AeroFlux Stage 2: A Scalable, Portable Big-Data Platform for Real-Time Flight-Delay Intelligence

Formal Project Proposal

Jonathan Wilson

2026-07-18

Course: AIT 614 Sec 001
Instructor: Dr. Duoduo Liao
Institution: George Mason University

1 Introduction

Flight delays are among the most costly and disruptive failures in the U.S. National Airspace System (NAS), and they rarely occur in isolation. A single late arrival propagates downstream through the same aircraft's rotation and through shared airport departure and arrival queues, so a local disturbance becomes a network-wide disruption. Existing airline and airport tools remain largely retrospective or single-source, reporting delays after they occur rather than anticipating them from the live state of the airspace.

Scope. AeroFlux Stage 2 builds a *scalable, modular, configurable, portable, and decoupled* big-data platform that ingests, fuses, processes, and analyzes aviation and weather data in real time. The primary use case is flight-delay prediction; the same architecture is designed to support congestion analysis, delay-propagation modeling, operational monitoring, decision support, and agentic AI. Stage 1 established a historical machine-learning pipeline and an interactive prototype (Wilson et al., 2026); Stage 2 adds the live data platform, a baseline-versus-advanced model comparison, and an explainable analyst agent.

Technical challenges. Building this platform requires solving several non-trivial problems:

(1) Ingesting heterogeneous, error-prone streaming sources (FAA SWIM XML, ADS-B, weather) without data loss;
(2) *Entity resolution* across systems that share no universal key—for example, the aircraft tail number is not published in SWIM flight data and must be recovered from ADS-B which itself has limitations due to lack of coverage;

(3) Unstable identifiers, since a flight's `flight_ref` changes on plan amendment while its Globally Unique Flight Identifier (GUFI) remains stable;
(4) Train-serve skew, because historical labels (Bureau of Transportation Statistics) and live features are measured differently; and
(5) Delivering all of this at demo quality on a portable, low-cost footprint.

Several of these challenges have already been partially addressed through a working end-to-end prototype pipeline. However, the proposed implementation is intended to reduce and manage these limitations rather than eliminate every possible source of missing data, ambiguity, or operational uncertainty.

2 Related Work

Machine-learning delay prediction. Most prior work frames delay prediction as supervised classification or regression over historical records. The review by Sternberg et al. (2017) describes the major approaches used for flight-delay prediction, while more recent work continues to evaluate machine-learning algorithms and class-imbalance handling (AlBassam & AlShahrani, 2025). These approaches can achieve strong accuracy on curated historical data, but they are often retrospective, single-source, and treat flights as independent observations, which limits their ability to represent the network coupling that drives real-world disruptions.

Delay-propagation modeling. A separate literature models how delays ripple through airline schedules. Beatty et al. (1999) introduced an early method for evaluating delay propagation through an airline schedule, Wong & Tsai (2012) applied a survival model, and Kafle & Zou (2016) proposed an analytical-econometric approach. These methods capture important propagation mechanisms but are generally offline and disconnected from real-time streaming prediction. Belcastro et al. (2016) demonstrated scalable data mining for flight-delay prediction, establishing the value of big-data tooling while leaving room for tighter integration with real-time data fusion.

Operational data fusion. The state of the art in operational systems includes NASA's Airspace Technology Demonstration 2 (ATD-2) *Fuser*, which fuses multiple aviation data sources into a unified flight record (National Aeronautics and Space Administration, 2021). Its strength is production-grade fused operational data; its limitation for this project is that it is not designed as a lightweight, open, extensible ML/AI experimentation platform.

Relevance of this approach. AeroFlux Stage 2 bridges these strands. It couples lightweight, Fuser-inspired real-time fusion and entity resolution with propagation-aware feature engineering and an explainable retrieval-augmented agent—integrating capabilities that prior work often treats separately.

3 Objectives

1. Ingest and fuse FAA SWIM, ADS-B, and weather data into a validated, per-flight canonical state in real time.

2. Deliver a portable, decoupled platform that runs unchanged on a laptop, school hardware, or any cloud VM (container-first, cloud-agnostic).
3. Compare a baseline model against an advanced, propagation- and weather-aware model for arrival-delay prediction.
4. Evaluate both **predictive** performance and **system** performance.
5. Provide a starter *Aviation Operations Analyst Agent* that retrieves evidence, explains model predictions, and recommends analytical next steps—without making operational decisions.
6. Achieve a demonstrable minimum implementation within approximately one month.

4 Datasets

The platform fuses four aviation and weather sources plus a documentation corpus, summarized in Table 1. FAA SWIM provides the authoritative live NAS flight state; ADS-B supplies the physical airframe identity (ICAO 24-bit address and registration) needed to link an aircraft's successive legs; NOAA METAR/TAF supplies weather, a leading delay driver; and the BTS On-Time Performance dataset—which includes actual gate times and tail numbers—provides historical training labels. Large-scale ADS-B research networks such as OpenSky demonstrate the usefulness of distributed aircraft surveillance data for aviation research (Schäfer et al., 2014).

Table 1: AeroFlux Stage 2 data sources, retention policies, and estimated storage requirements. Estimated sizes are project-planning ranges rather than published source totals and depend on geographic scope, ingestion frequency, selected fields, compression, and duplicate handling.

Source	Type	Role	Retention	Estimated retained size
FAA SWIM (TFMS)	Streaming XML	Live flight state (primary)	48 h	Approximately 5–25 GB
ADS-B (airplanes.live / adsb.lol)	Streaming JSON	Airframe and tail resolution	48 h	Approximately 2–10 GB
NOAA METAR / TAF	Polled JSON / text	Weather observations and forecasts	48 h	Approximately 50–250 MB
BTS On-Time Performance	Historical CSV / Parquet	Historical training labels	Persistent	Approximately 20–40 GB as CSV or 4–10 GB as Parquet

Source	Type	Role	Retention	Estimated retained size
Aviation documents (FAA, SWIM dictionaries, and related references)	PDF / HTML / text	RAG documentation corpus	Persistent	Approximately 0.5–2 GB, including extracted text and embeddings

A raw SWIM message (abridged) is shown in Listing 1; the corresponding fused, validated canonical record produced by the ingestion pipeline is shown in Listing 2. Each SWIM document explodes into one record per flight, which the pipeline fuses by GUFI across many messages over time.

Listing 1 Raw SWIM TFMS flight message (abridged).

```
<fdm:fltdMessage acid="AAL2033" airline="AAL" arrArpt="KLGA" depArpt="KCLT"
   flightRef="150976233" msgType="FlightModify" ...>
  <nxcm:flightStatus>PLANNED</nxcm:flightStatus>
  <nxcm:aircraftModel>A320</nxcm:aircraftModel>
  <nxcm:flightTimeData airlineOutTime="2026-07-04T15:39:00Z"
                       airlineInTime="2026-07-04T17:35:00Z"/>
</fdm:fltdMessage>
```

Listing 2 Fused, schema-validated canonical flight-instance record.

```
{ "flight_instance_id": "KN6770564K", "callsign": "AAL2033",
  "flight_number": "AA2033", "carrier_name": "American Airlines",
  "origin": "KCLT", "destination": "KLGA", "aircraft_type": "A320",
  "scheduled_gate_departure": "2026-07-04T15:39:00Z",
  "estimated_arrival": "2026-07-04T17:41:00Z", "flight_status": "PLANNED",
  "resolution_status": "airline_resolved", "schema_version": "1.0" }
```

5 Methodology and Proposed Solution

Pipeline. Ingestion services publish each source to a dedicated Kafka topic (48-hour retention). Apache Spark Structured Streaming consumes these topics and applies the existing `aeroflux_parser` to parse, normalize, validate against a strict schema, and fuse messages into per-flight records via GUFI-keyed, source-priority mediation, with `flight_ref`. We us this only as a reconciliation bridge and ADS-B used to resolve the tail. Fused records land in MongoDB (silver, 48-hour TTL) and, together with BTS (historical used for training) backfill, in a MinIO data lake (gold parquet). The full architecture is shown in Figure 1.

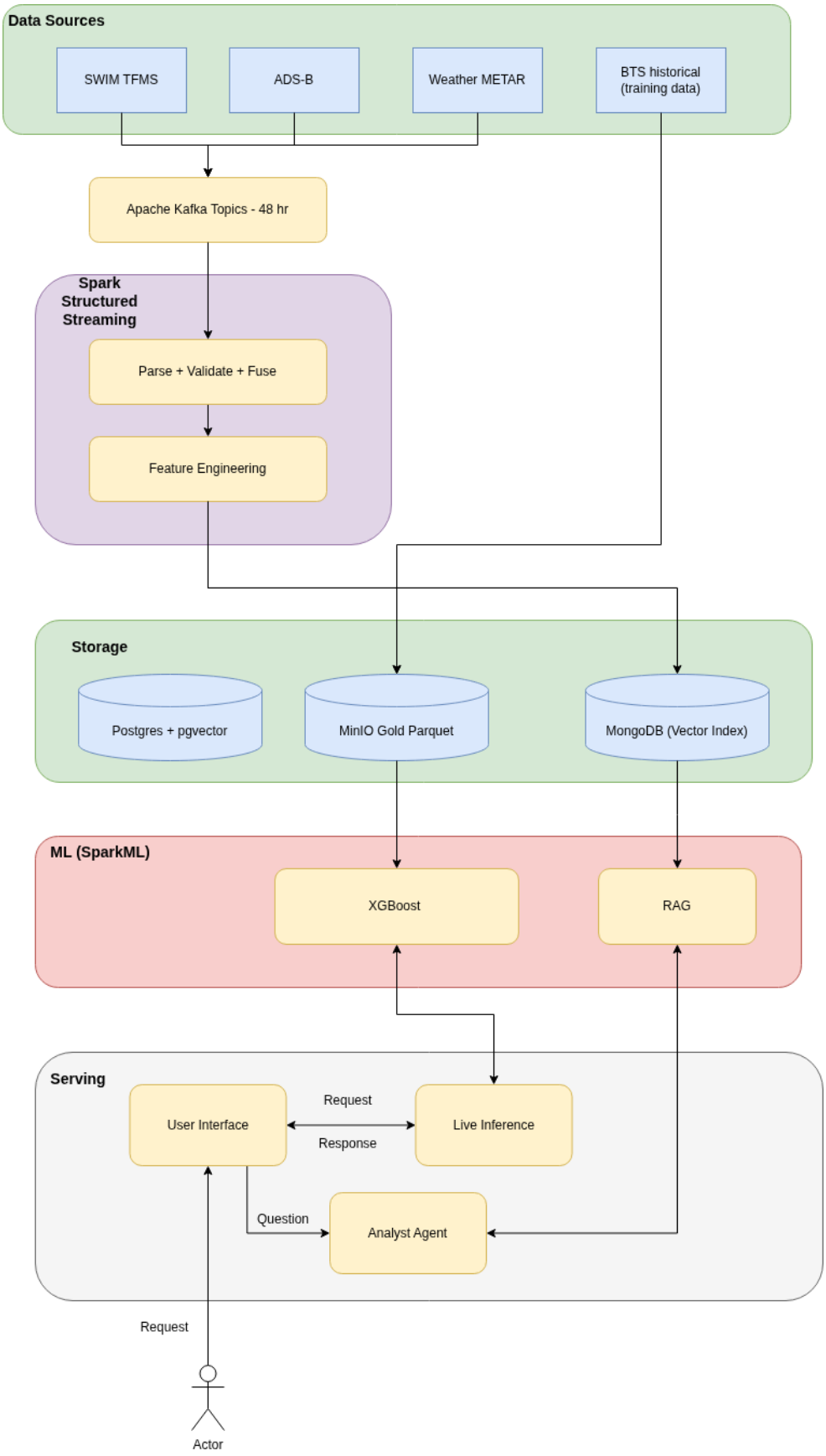

Figure 1: AeroFlux Stage 2 system architecture.

Modeling. The main model is a gradient-boosted tree using XGBoost (Chen & Guestrin, 2016) and is enriched with *propagation* features (previous-leg arrival delay and scheduled turnaround buffer), weather features, and rolling airport demand. This design directly operationalizes findings from the delay- propagation literature (Beatty et al., 1999; Kafle & Zou, 2016). The model targets a 15-minute arrival-delay classification, with class imbalance addressed through resampling, a concern emphasized in recent delay-prediction research (AlBassam & AlShahrani, 2025). Models are tracked in MLflow and explained with SHAP (Lundberg & Lee, 2017), whose outputs feed the agent.

Add possible section on how we could leverage distributed training See: https://www.ischool.berkeley.edu/projects/2025/air-travel-delay-prediction-feature-engineering-and-ml-approaches This would be a bonous as the model is already trained. This project focus on how to use the model during live inference of real time data feeds.

Evaluation. Both predictive and system performance are evaluated (Table 2). Predictive metrics include ROC-AUC, PR-AUC, F1, and Brier score and calibration, using point-in-time features to prevent leakage. System metrics include end-to-end latency, sustained message throughput, fusion completeness (fields populated per flight), and resource footprint.

Table 2: Baseline-versus-advanced comparison and evaluation plan.

Dimension	Baseline	Advanced	Metrics
Model	???	XGBoost + propagation + weather	ROC-AUC, PR-AUC, F1, Brier
Features	Flight-level	+ propagation, weather, demand	Feature importance (SHAP)
System	—	Streaming pipeline	Latency, throughput, completeness

(Bonous if we have time) Agentic AI and LLM approach. The starter *Aviation Operations Analyst Agent* answers natural-language questions (for example, "Why is this flight likely delayed?") by combining retrieval-augmented generation (Lewis et al., 2020) over the aviation-documentation corpus (embeddings in pgvector) with tool calls that query the fused databases, invoke the delay model, and return SHAP explanations. The agent retrieves evidence, explains predictions, and recommends next steps; it does **not** execute operational aviation decisions.

6 Development Environment

The stack (Table 3) is entirely open-source and container-first. Every component runs as a Docker Compose service, so the platform deploys unchanged on a laptop, university hardware, or a single cloud VM; managed AWS services (MSK, EMR, SageMaker) are an optional scale path rather than a requirement. MinIO provides an S3-compatible object store that swaps to

real S3 with no code change. Short retention keeps storage and cost minimal (target budget: a modest VM plus a few dollars of LLM API usage).

Table 3: Development environment and technology stack.

Category	Tools
Language	Python >3.11
Streaming	Apache Kafka, Spark Structured Streaming
Storage	MongoDB, PostgreSQL + pgvector, MinIO (S3-compatible)
ML	SparkML, scikit-learn, XGBoost (Chen & Guestrin, 2016), MLflow, SHAP (Lundberg & Lee, 2017)
(optional) LLM / RAG	Claude API, pgvector, tool-calling agent loop
Serving / UI	FastAPI, Dash / Plotly
(Optional) Orchestration	Prefect or Airflow
Packaging	Docker + Docker Compose
Platform	Hybrid - On-prem (streaming) and AWS S3, EC2; Optional EMR for scaling

7 Project Tasks and Timeline

The minimum implementation is scoped to approximately one month (Table 4), building on Stage 1's completed ingestion, fusion, entity-resolution, schema contract, and baseline model. Future capabilities—graph analytics, simulation, digital twin, and a multi-agent system—are explicitly out of scope for this timeline.

Table 4: One-month project timeline and effort allocation.

Week	Tasks	Est. hours
1	Add weather source; Spark Structured Streaming pipeline; MongoDB silver store + TTL	20
2	Feature engineering (propagation, demand, weather); gold dataset; baseline vs. advanced training; MLflow; evaluation	22
3	FastAPI inference API + SHAP; pgvector RAG; analyst agent + tools	22

Week	Tasks	Est. hours
4	Integration, UI wiring, system + predictive evaluation, demo, write-up; buffer	18

References

AlBassam, S. A. A., & AlShahrani, D. N. (2025). Flight delay prediction: Evaluating machine learning algorithms for enhanced accuracy. *PLOS ONE*, *20*(12), e0335141.

Beatty, R., Hsu, R., Berry, L., & Rome, J. (1999). Preliminary evaluation of flight delay propagation through an airline schedule. *Air Traffic Control Quarterly*, *7*(4), 259–270.

Belcastro, L., Marozzo, F., Talia, D., & Trunfio, P. (2016). Using scalable data mining for predicting flight delays. *ACM Transactions on Intelligent Systems and Technology*, *8*(1), 1–20.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

Kafle, N., & Zou, B. (2016). Modeling flight delay propagation: A new analytical-econometric approach. *Transportation Research Part B: Methodological*, *93*, 520–542.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems 33*, 9459–9474.

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems 30*, 4765–4774.

National Aeronautics and Space Administration. (2021). *Airspace technology demonstration 2 (ATD-2) fuser*. Computer software; GitHub. https://github.com/nasa/atd2-fuser

Schäfer, M., Strohmeier, M., Lenders, V., Martinovic, I., & Wilhelm, M. (2014). Bringing up OpenSky: A large-scale ADS-B sensor network for research. *Proceedings of the 13th International Symposium on Information Processing in Sensor Networks*, 83–94.

Sternberg, A., Soares, J., Carvalho, D., & Ogasawara, E. (2017). *A review on flight delay prediction*. https://arxiv.org/abs/1703.06118

Wilson, J., Nindra, K., Austin, I., & Sokolow, V. (2026). *From prediction to digital twin: A propagation-aware machine learning framework for flight delay modeling*. https://jonathanwilsonami.github.io/OR568_ML_Project/

Wong, J.-T., & Tsai, S.-C. (2012). A survival model for flight delay propagation. *Journal of Air Transport Management*, *23*, 5–11.